

Apuntes Semana 9: Clasificación con Redes Neuronales y Proyecto I – Reconocimiento de Comandos de Voz

Allan Bolaños Barrientos
Escuela de Ingeniería en Computación
Instituto Tecnológico de Costa Rica
2024145458

Resumen—En estos apuntes se repasan conceptos introductorios de clasificación con redes neuronales a partir del caso de MNIST. Se revisa el paso de regresión logística binaria a clasificación multiclase, el uso de one-hot encoding, la representación matricial de una capa y la motivación para usar capas intermedias no lineales en redes neuronales. Además, se resumen los lineamientos del Proyecto I del curso, incluyendo el uso del dataset *Speech Commands* [1], la técnica de aumento de datos SpecAugment [2], el diseño de modelos convolucionales, el preprocesamiento mediante espectrogramas, la evaluación de resultados y la exportación del modelo a ONNX para su integración en una aplicación móvil. Se incluyen también resúmenes de las lecturas asignadas [3].

Index Terms—redes neuronales, clasificación, MNIST, one-hot encoding, CNN, keyword spotting, SpecAugment, ONNX

I. REPASO: CLASIFICACIÓN DE MNIST

El dataset MNIST consiste en muestras de dígitos escritos a mano, donde cada imagen está representada por una matriz de píxeles. Tomando cada píxel como un *feature*, es posible construir una regresión logística que realice clasificación binaria de un dígito específico.

La regresión logística se compone de tres elementos fundamentales:

Parte lineal, que modela la interacción entre los features y los pesos del modelo:

$$f_{w,b}(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b \quad (1)$$

Parte no lineal, dada por la función sigmoide, que transforma la salida lineal en una probabilidad entre 0 y 1:

$$g(x) = \frac{1}{1 + e^{-x}} \quad (2)$$

Anidación de funciones, donde primero se ejecuta la transformación lineal y su resultado se pasa a la función sigmoide:

$$h(x) = g(f(x)) \quad (3)$$

Esta composición permite al modelo tomar decisiones binarias (pertenece o no a la clase) a partir de entradas de alta dimensionalidad como imágenes.

II. SISTEMA MULTINOMIAL

Un sistema multinomial extiende la clasificación binaria para manejar **múltiples clases** de forma simultánea. En lugar de responder si una entrada pertenece o no a una clase, el modelo debe determinar a cuál de N clases posibles pertenece.

II-A. Solución: One-Hot Encoding

Para representar múltiples clases se utiliza la codificación *One-Hot*: cada clase se representa como un vector binario de longitud N , donde únicamente la posición correspondiente a la clase correcta toma el valor 1 y todas las demás toman el valor 0. Esto elimina cualquier relación ordinal artificial entre las clases.

Aplicado a MNIST (dígitos del 0 al 9), la solución consiste en entrenar 10 regresiones logísticas en paralelo, cada una especializada en reconocer un dígito específico. La Tabla I muestra la codificación resultante.

Cuadro I
CODIFICACIÓN ONE-HOT PARA DÍGITOS DEL 0 AL 9

Clase	Codificación One-Hot
0	1000000000
1	0100000000
2	0010000000
3	0001000000
4	0000100000
5	0000010000
6	0000001000
7	0000000100
8	0000000010
9	0000000001

II-B. Escalamiento con la cantidad de píxeles

Con un solo píxel de entrada, cada regresión logística recibe un único valor con su peso w y su *bias* b . Al aumentar la cantidad de píxeles el sistema escala rápidamente: con 784 píxeles la representación gráfica se vuelve imposible, pero el principio se mantiene: toda la información de entrada se distribuye a todas las regresiones logísticas para que cada una

tome su decisión de forma independiente. Es como pasar una fotocopia de la imagen a cada neurona para que decida si sabe reconocerla.

III. CONVERSIÓN DE VECTOR A MATRIZ

Calcular cada regresión logística de forma individual requiere N operaciones separadas. Para hacer esto eficiente se recurre al **álgebra lineal**: se reemplazan los vectores de pesos individuales por una matriz de pesos W , permitiendo computar toda la capa en una sola operación matricial:

$$Z = XW + b \quad (4)$$

donde X es el vector de entrada, $W \in \mathbb{R}^{784 \times 10}$ contiene los pesos de todas las regresiones logísticas para MNIST, y b es el vector de *bias*. Esta representación es la base del cómputo eficiente en hardware moderno como GPUs.

Cabe destacar que una entrada lineal conectada con 10 regresiones logísticas sigue siendo un *problema lineal*: se define una frontera de decisión lineal en el espacio de entrada, lo cual es insuficiente para resolver escenarios no lineales.

IV. CLASIFICACIÓN CON CAPAS INTERMEDIAS

La clave para introducir no linealidad está en aplicar la función sigmoide a la salida de una capa intermedia. Al hacerlo, los valores de salida quedan en el rango $[0, 1]$ y la transformación pasa a ser no lineal. Por ejemplo, para clasificar si una imagen es el dígito 5: en lugar de usar una sola regresión logística directamente, se pasa la entrada a una capa con múltiples neuronas, y esa salida (ya no lineal) se alimenta a la siguiente capa.

Las capas del sistema se identifican como:

- **Capa de entrada:** recibe los features originales; su tamaño está determinado por la dimensionalidad del problema (784 para MNIST).
- **Capa intermedia:** realiza el cómputo entre la entrada y la salida; introduce no linealidad al sistema.
- **Capa de salida:** produce la decisión final, ya sea binaria (sigmoide) o multinomial.

V. REDES NEURONALES

Una red neuronal es la generalización de este sistema de capas: múltiples capas intermedias apiladas, cada una introduciendo no linealidad adicional, permiten al modelo aprender representaciones jerárquicas de los datos.

Las características fundamentales de una red neuronal son:

- **Profundidad:** entre más capas intermedias tenga la red, más complejos son los problemas que puede resolver. Es un hiperparámetro del modelo.
- **Diversidad entre capas:** cada capa debe aprender representaciones distintas para que la profundidad sea útil.
- **Optimizabilidad:** siempre que las funciones de activación sean derivables, es posible aplicar descenso de gradiente para optimizar todos los pesos mediante *back-propagation*.
- **Neuronas:** cada capa está compuesta por unidades de cómputo individuales con sus propios pesos y *bias*.

Una posible solución *fully connected* para MNIST puede utilizar dos capas intermedias y una capa final multinomial que clasifica los 10 dígitos.

VI. PROYECTO I: RECONOCIMIENTO DE COMANDOS DE VOZ

VI-A. Descripción General

El proyecto consiste en diseñar e implementar una solución completa de inteligencia artificial para reconocimiento de comandos de voz, utilizando el *Speech Commands Dataset v0.02* [1]. El modelo final debe integrarse en una aplicación Android o iOS mediante ONNX Runtime Mobile, realizando inferencia localmente sin depender de servicios externos.

VI-B. Comandos Autorizados

El sistema debe reconocer exactamente 10 comandos, mostrados en la Tabla II.

Cuadro II
LISTA DE COMANDOS AUTORIZADOS

Comando	Caso de uso
yes / no	Confirmación y cancelación
up / down / left / right	Navegación en la interfaz
on / off	Control de dispositivos
stop / go	Control de rutinas

VI-C. Concepto de Aplicación Móvil

La aplicación debe construirse como una interfaz de navegación por voz inspirada en un escenario de *Smart Home*. Debe incluir al menos las siguientes pantallas:

- **Dashboard:** pantalla principal con tarjetas (luces, ventilador, TV, rutinas). Se navega con up/down/left/right y se abre una sección con yes.
- **Control de dispositivo:** permite encender o apagar con on/off; yes confirma y no regresa.
- **Rutinas:** inicia/detiene rutinas con go/stop; navega entre ellas con up/down.
- **Confirmación:** aparece ante acciones sensibles; requiere yes para ejecutar o no para cancelar.

La app no debe depender de servicios externos; la inferencia se ejecuta localmente mediante ONNX Runtime Mobile.

VI-D. Dataset y Preprocesamiento

Se utilizará el *Speech Commands Dataset v0.02* [1]. Dado que las CNN reciben imágenes como entrada, los audios deben transformarse en **espectrogramas**: representaciones 2D donde el eje horizontal es el tiempo, el eje vertical la frecuencia, y la intensidad del color la amplitud de cada componente frecuencial.

Se deben preparar dos conjuntos de datos:

- **Conjunto crudo:** audios convertidos directamente a espectrogramas sin modificación adicional. Representa la línea base del experimento.

- **Conjunto aumentado:** igual al anterior pero con técnicas de *data augmentation* inspiradas en audio [2]. Se recomienda **SpecAugment**, que aplica enmascaramiento aleatorio de bandas de frecuencia y segmentos de tiempo directamente sobre el espectrograma. Su uso debe estar justificado con literatura en el informe.

Cada conjunto se divide en tres particiones: **entrenamiento**, **validación** y **testing**. La partición de testing se reserva exclusivamente para la evaluación final del modelo seleccionado.

Nota práctica: en Google Colab se tarda aproximadamente 20 minutos por entrenamiento. El entrenamiento no debe realizarse con CPU; se recomienda GPU.

VI-E. Diseño de Arquitectura

Se deben construir manualmente dos modelos con PyTorch, utilizando únicamente `torch.nn`:

Modelo A — LeNet-5 clásico: variante adaptada a la clasificación de audio a partir de representaciones espectrales. El tamaño de entrada mínimo recomendado es de 128. Se permite modificar tamaño de entrada, número de canales e hiperparámetros; cualquier otra modificación debe consultarse con el profesor.

Modelo B — Arquitectura alternativa: cualquier arquitectura distinta basada en literatura (EfficientNet, MobileNet, Inception, DenseNet, ResNet [3]) o diseño propio fundamentado académicamente. Debe considerar el contexto de despliegue en dispositivo móvil con recursos limitados. No puede ser un LeNet-5 aumentado; debe demostrarse por qué es diferente y funcional.

VI-F. Entrenamiento

Se deben entrenar cuatro modelos base:

- Modelo A / Datos crudos
- Modelo A / Datos aumentados
- Modelo B / Datos crudos
- Modelo B / Datos aumentados

Para cada combinación se realizan al menos 3 entrenamientos con distintos hiperparámetros (tasa de aprendizaje, tamaño de *batch*, número de épocas, optimizador), resultando en un mínimo de **12 modelos entrenados** en total (4 combinaciones $\times$ 3 configuraciones de hiperparámetros). Para cada entrenamiento se deben mostrar las curvas de *loss* y métricas del conjunto de entrenamiento y validación para verificar ausencia de *overfitting* o *underfitting*.

VI-G. Evaluación y Comparación de Modelos

Para la comparación se debe utilizar **Weights & Biases (WandB)**, que permite trazar y visualizar en tiempo real el proceso de entrenamiento. Las métricas a registrar incluyen: *loss* de entrenamiento y validación, *accuracy*, F1-Score y matriz de confusión.

La selección del mejor modelo se realiza en dos etapas: primero se selecciona el mejor modelo dentro de cada combinación con base en el conjunto de test; luego se comparan los cuatro mejores modelos para determinar el modelo final a desplegar.

VI-H. Exportación a Móvil con ONNX

Los pasos para el despliegue son:

1. **Congelar el modelo:** colocar en modo inferencia y conservar pesos entrenados.
2. **Definir el pipeline de entrada:** documentar formato de audio, frecuencia de muestreo, preprocesamiento y forma del tensor de entrada.
3. **Exportar a ONNX:** verificar que entradas y salidas coincidan con el modelo original mediante ejemplos de prueba.
4. **Integrar en la app:** cargar el `.onnx` con ONNX Runtime Mobile, reproduciendo el mismo preprocesamiento del entrenamiento.

VI-I. Entrega

La entrega es un archivo `.zip` con:

- Código fuente del informe en L^AT_EX usando plantilla IEEE.
- PDF del informe con visualizaciones e interpretación de resultados.
- Jupyter Notebook con el código completo del modelo.
- Código fuente de la aplicación móvil integrada con los comandos de IA.

VI-J. Rúbrica de Evaluación

La Tabla III muestra los criterios de evaluación del proyecto.

Cuadro III
RÚBRICA DE EVALUACIÓN DEL PROYECTO I

Criterio	Puntaje Máx.
Diseño y arquitectura Modelo A	5
Diseño y arquitectura Modelo B	10
Selección de técnicas de aumentación	10
Entrenamiento Modelo A (crudo y aumentado)	15
Entrenamiento Modelo B (crudo y aumentado)	15
Métricas y selección mejor modelo A (test)	10
Métricas y selección mejor modelo B (test)	10
Comparación de modelos finales	10
Integración en aplicación móvil	15
Total	100

VII. RESUMEN DE LECTURAS ASIGNADAS

VII-A. *Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition* [1]

Este trabajo presenta el dataset *Speech Commands*, diseñado específicamente para entrenar y evaluar sistemas de *keyword spotting* en dispositivos. A diferencia de los datasets de reconocimiento de voz general (LibriSpeech, Common Voice), este se enfoca en el reconocimiento de palabras aisladas bajo restricciones de recursos computacionales y energéticos propios de dispositivos móviles.

El dataset fue recolectado mediante una aplicación web open-source que usaba la WebAudioAPI. Los audios se capturaron a través de micrófonos de teléfono o laptop en ambientes variados, buscando diversidad de acentos y condiciones de ruido. Cada grabación tiene una duración estándar de un segundo.

El proceso de control de calidad combinó filtros automáticos (tamaño del archivo OGG comprimido) con revisión humana vía *crowdsourcing*.

La versión 2 del dataset (v0.02) contiene **105,829 grabaciones de 35 palabras** realizadas por 2,618 hablantes distintos. El vocabulario core incluye los dígitos del cero al nueve, y diez palabras de comando relevantes para IoT: *Yes, No, Up, Down, Left, Right, On, Off, Stop* y *Go*. Estos últimos son precisamente los 10 comandos que se usarán en el Proyecto I.

Para la evaluación, el dataset incluye los archivos `validation_list.txt` y `testing_list.txt`, que definen las particiones mediante una función *hash* sobre el nombre del archivo. Esto garantiza que los archivos permanezcan en la misma partición entre versiones del dataset, evitando contaminación cruzada. La métrica principal es el **Top-One Error**, que mide la proporción de predicciones incorrectas. El modelo baseline del tutorial de TensorFlow alcanza un 88.2 % de exactitud en esta métrica.

VII-B. SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition [2]

SpecAugment es una técnica de aumento de datos que opera directamente sobre el log-mel espectrograma del audio, tratándolo como si fuera una imagen. Su principal ventaja es que es simple, computacionalmente barata y no requiere datos adicionales, por lo que puede aplicarse *online* durante el entrenamiento.

La política de aumentación consiste en tres tipos de deformaciones:

1. **Time warping:** deformación de la serie temporal en dirección horizontal. Un punto aleatorio dentro del espectrograma es desplazado a izquierda o derecha por una distancia w tomada de una distribución uniforme $[0, W]$.
2. **Frequency masking:** se enmascaran f canales de frecuencia mel consecutivos $[f_0, f_0 + f)$, donde f se elige uniformemente en $[0, F]$ y f_0 en $[0, \nu - f)$.
3. **Time masking:** se enmascaran t pasos de tiempo consecutivos $[t_0, t_0 + t)$, donde t se elige uniformemente en $[0, T]$ y t_0 en $[0, \tau - t)$.

Los valores enmascarados se ponen a cero, lo que equivale al valor medio del espectrograma normalizado. Es posible aplicar múltiples máscaras de frecuencia y tiempo; estas pueden solaparse. Los autores proponen varias políticas predefinidas (LB, LD, SM, SS) con distintos niveles de agresividad.

Un hallazgo clave del trabajo es que la aumentación **convierte un problema de overfitting en uno de underfitting**, lo que permite mejorar el rendimiento usando redes más grandes y entrenando por más tiempo. SpecAugment logra resultados estado del arte en LibriSpeech 960h (6.8 % WER sin modelo de lenguaje) y Switchboard 300h, superando sistemas híbridos más complejos.

Para el Proyecto I, el uso de SpecAugment está particularmente justificado ya que el dataset *Speech Commands* tiene un tamaño moderado y las CNNs propuestas pueden beneficiarse del enmascaramiento de frecuencias y tiempo para mejorar su generalización. *Time warping* es la transformación más costosa

y con menor impacto; puede omitirse si hay restricciones de tiempo de entrenamiento.

VII-C. Training Keyword Spotters with Limited and Synthesized Speech Data [3]

Este paper de Google Research explora la efectividad del habla sintetizada para entrenar modelos pequeños de *keyword spotting* (aproximadamente 400k parámetros), en el contexto de dispositivos con recursos muy limitados (procesadores DSP).

La propuesta arquitectónica divide el modelo en dos partes:

- **Embedding model:** contiene 5 bloques convolucionales ($\sim 330k$ pesos) y fue entrenado con 200 millones de clips de audio de YouTube sobre 5000 palabras clave agrupadas en 125 grupos de 40. Su objetivo es extraer representaciones útiles para la detección de palabras clave arbitrarias. Está disponible públicamente en TensorFlow Hub.
- **Head model:** contiene un solo bloque convolucional más un bloque de clasificación ($\sim 55k$ pesos). Se entrena sobre el embedding del modelo anterior en pocos minutos, y escala eficientemente con el número de palabras clave (solo 96 pesos adicionales por nueva palabra, frente a 1536 de un modelo completamente conectado).

Los resultados más relevantes para el Proyecto I son:

- Un modelo entrenado con datos sintéticos únicamente (3220 ejemplos) alcanza un 92.6 % de precisión usando el *head model*, comparado con 56.7 % del modelo completo en las mismas condiciones.
- Reemplazar apenas el 50 % de los datos sintéticos con datos reales cierra casi el 90 % de la brecha de rendimiento entre usar solo datos reales y solo datos sintéticos.
- El *head model* con 50 ejemplos reales por palabra es equivalente en precisión al modelo completo entrenado con más de 4000 ejemplos reales.

Este paper es relevante como referencia de arquitectura y de la estructura del informe (especialmente la sección de *related work*), y demuestra que arquitecturas convolucionales ligeras son viables para *keyword spotting* en dispositivos móviles, lo cual es exactamente el caso de uso del Proyecto I.

REFERENCIAS

- [1] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," *arXiv preprint arXiv:1804.03209*, Apr. 2018.
- [2] D. S. Park, W. Chan, Y. Zhang, C.-C. Chiu, B. Zoph, E. D. Cubuk, and Q. V. Le, "SpecAugment: A simple data augmentation method for automatic speech recognition," *arXiv preprint arXiv:1904.08779*, 2019.
- [3] J. Lin, K. Kilgour, D. Roblek, and M. Sharif, "Training keyword spotters with limited and synthesized speech data," in *Proc. ICASSP*, 2020. arXiv:2002.01322.
- [4] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [5] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. CVPR*, 2016.